

Web Technologies 1 (Frontend)

Week 4: CSS Layouts — Flexbox & Grid

Dossymzhan

1 Overview

Week 4 covers the `display` property, the difference between hiding elements, the two main modern layout systems — Flexbox (1D) and Grid (2D) — their terminology, key properties, and how to combine them.

2 1. The `display` Property

`display` controls how an element is shown on the page and how it interacts with its neighbors.

- Default display: `block` or `inline`.
- The default can be changed, e.g. `display: inline` on `<li>` for a horizontal menu.
- Flexbox and Grid are turned on via `display: flex` and `display: grid`.

Block-level elements: start on a new line, take full width. Examples: `<div>`, `<h1>`–`<h6>`, `<p>`, `<form>`, `<header>`, `<footer>`, `<section>`.

Inline elements: stay in the same line, take only the needed width. Examples: `<span>`, `<a>`, `<img>`.

3 2. `display: none` vs `visibility: hidden`

- `display: none` — element is removed; it takes up no space.
- `visibility: hidden` — element is hidden but its space is kept in the layout.

4 3. Flexbox

The CSS Flexible Box Layout Module (Flexbox) is a **one-dimensional** layout method for arranging items in a row or a column, without floats or manual positioning. Items grow to fill free space or shrink to fit the container.

With Flexbox you can:

- center content horizontally and vertically;
- distribute space evenly between items;
- make columns the same height regardless of content;
- change the order/direction of items without editing the HTML.

4.1 Flexbox Terminology

- **Flex container** — the parent with `display: flex`.
- **Flex items** — the direct children inside the container.
- **Main axis** — the primary axis along which flex items are laid out.
- **Cross axis** — perpendicular to the main axis.
- **main-start** / **main-end** — the starting/ending edges of the main axis.
- **main size** — total size of flex items along the main axis (width if row, height if column).
- **cross-start** / **cross-end** — starting/ending edges of the cross axis.
- **cross size** — total size of flex items along the cross axis.

4.2 Key Flexbox Properties — Container

Property	Description
<code>display</code>	Defines a flex container: <code>flex</code> or <code>inline-flex</code>
<code>flex-direction</code>	Main axis: <code>row</code> , <code>row-reverse</code> , <code>column</code> , <code>column-reverse</code>
<code>flex-wrap</code>	Wrapping: <code>nowrap</code> , <code>wrap</code> , <code>wrap-reverse</code>
<code>flex-flow</code>	Shorthand for <code>flex-direction</code> + <code>flex-wrap</code>
<code>justify-content</code>	Aligns items on the main axis (horizontally by default)
<code>align-items</code>	Aligns items on the cross axis (vertically by default)
<code>align-content</code>	Aligns flex lines when items wrap

4.3 Key Flexbox Properties — Items

Property	Description
<code>order</code>	Order in which items appear (default 0)
<code>flex-grow</code>	How much an item can grow relative to others (default 0)
<code>flex-shrink</code>	How much an item can shrink relative to others (default 1)
<code>flex-basis</code>	Initial size before growth/shrink (default <code>auto</code>)
<code>flex</code>	Shorthand for <code>flex-grow</code> , <code>flex-shrink</code> , <code>flex-basis</code>
<code>align-self</code>	Overrides <code>align-items</code> for a single item

4.4 flex-direction and justify-content

```
.flex-container {  
  display: flex;  
  flex-direction: column; /* row (default) | row-reverse | column |  
    column-reverse */  
}
```

justify-content values: flex-start (default), center, flex-end, space-between, space-around, space-evenly.

4.5 Flexbox Examples

- **Equal share:** flex: 1 on every item — each takes an equal share of available space.
- **Auto margins:** margin-left: auto; pushes an item to the edge (e.g., a "Log in / Sign up" block pushed right in a navbar).
- **Column layouts:** flex-direction: column; stacks items vertically (e.g., a card with the button pinned to the bottom via margin-top: auto;).
- **Non-shrinking items:** flex-shrink: 0; keeps an item (e.g., an avatar) at a fixed size while sibling text wraps.
- **Wrapping:** flex-wrap: wrap; moves overflowing items to the next line.
- **Stretch alignment:** align-items: stretch; equalizes item heights.

```
.card { flex-direction: column; }  
.card .btn { margin-top: auto; }  
.avatar { flex-shrink: 0; }  
.buttons { margin-left: auto; }
```

5 4. Grid Layout

CSS Grid Layout is a **two-dimensional** grid-based system with rows *and* columns, making it easier to build complex layouts without floats or positioning.

With CSS Grid you can:

- define fixed, flexible, or proportional track sizes (fr, auto, %);
- place items into specific cells or span them across multiple rows/columns, regardless of source order;
- build complex layouts (dashboards, galleries, page templates) with consistent alignment.

5.1 Grid Terminology

- **Grid Container** — parent with `display: grid`.
- **Grid Items** — direct children inside the grid.
- **Grid Track** — a row or a column.
- **Grid Area** — a rectangular space spanning multiple cells.
- **Grid Line** — dividing lines between cells.
- **Grid Cell** — a single "unit" of the grid.
- **Gap** — space between columns/rows.

5.2 Special Units and Functions

Unit/Function	Description
<code>fr</code>	Fraction of available space in the container (e.g. <code>1fr 2fr</code>)
<code>auto</code>	Size adjusts automatically to fit content
<code>min-content</code>	Track shrinks to the smallest size needed
<code>max-content</code>	Track expands to the largest size needed
<code>minmax(min, max)</code>	Track resizes between a min and max value
<code>fit-content(v)</code>	Track grows to fit content up to a maximum
<code>repeat(n, value)</code>	Repeats a track definition <i>n</i> times (e.g. <code>repeat(3, 1fr)</code>)

5.3 Key Grid Properties — Container

Property	Description
<code>grid-template-columns</code>	Number and width of columns
<code>grid-template-rows</code>	Number and height of rows
<code>gap / row-gap / column-gap</code>	Spacing between tracks
<code>justify-items / align-items</code>	Aligns items horizontally/vertically within cells
<code>justify-content</code>	Aligns the entire grid along the row axis
<code>align-content</code>	Aligns the entire grid along the column axis
<code>grid-template-areas</code>	Defines the layout using named areas

5.4 Key Grid Properties — Items

Property	Description
<code>grid-column</code>	Column(s) an item should occupy
<code>grid-row</code>	Row(s) an item should occupy
<code>grid-area</code>	Item position by row/column, or a named area
<code>justify-self</code>	Aligns a single item horizontally in its cell
<code>align-self</code>	Aligns a single item vertically in its cell

5.5 Spanning and Auto-fit

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(110px, 1fr));
  gap: 10px;
}
.lead {
  grid-column: span 2; /* item takes 2 columns */
  grid-row: span 2;    /* item takes 2 rows    */
}
```

`grid-column: 1 / 4;` means "from line 1 to line 4". `repeat(auto-fit, minmax(110px, 1fr))` makes the number of columns follow the window width.

5.6 Grid Template Areas

```
.page {
  display: grid;
  grid-template-columns: 120px 1fr;
  grid-template-rows: 50px 1fr 40px;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 10px;
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }
```

This names each grid area directly in the CSS layout string, giving a readable "map" of the page (header / sidebar + main / footer).

6 5. Flexbox vs Grid

Feature	Flexbox (1D)	Grid (2D)
Layout direction	Row <i>or</i> Column	Rows <i>and</i> Columns
Use case	Navbars, simple alignment	Complex page layouts
Approach	Content-first	Layout-first
Responsiveness	Easy with <code>flex-wrap</code>	Easy with <code>auto-fit</code> / <code>auto-fill</code>

6.1 When to Use Which

- **Use Flexbox for:** navbars, menus, buttons, form controls, one-direction layouts.

- **Use Grid for:** full page layout, dashboards, galleries, complex responsive designs.

7 6. Combining Flexbox and Grid

They work well together:

- Grid for the large-scale page layout; Flexbox for aligning content inside individual grid cells.
- A grid item can itself be a flex container.
- A flex item can itself be a grid container.

```
/* page layout: Grid */
.page {
  display: grid;
  grid-template-columns: 120px 1fr;
  grid-template-rows: 60px 1fr 40px;
  gap: 10px;
}

/* inside cells: Flexbox */
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.main {
  display: flex;
  gap: 10px;
}
.card { flex: 1; }
```

8 Summary

Week 4 covers the `display` property and how it differs from visibility, then the two modern CSS layout systems: one-dimensional **Flexbox** (container/item terminology, main/cross axis, `flex-direction`, `justify-content`, growing/shrinking items) and two-dimensional **Grid** (tracks, areas, lines, cells, `fr` units, `repeat()`/`minmax()`, `grid-template-areas`), when to use each, and how to combine them for real page layouts.